C PROGRAMMING • STUDY GUIDE

POINTERS

Complete Theory, Syntax, Memory Diagrams & Solved Programs

For University Exams • Competitive Exams • Teacher Recruitment Tests (TET/TGT/PGT/NET)

A concise yet comprehensive revision resource covering all 16 syllabus topics on Pointers in C

Table of Contents

1. Concept of Pointers
2. Pointer as an Address Variable
3. Concept of a Pointer Data Type and its Syntax
4. Built-in Address Operator (&)
5. Pointers to Existing Variables of Different Data Types and Their Uses
6. Use of Indirection Operator (*)
7. The Name of the Array as a Pointer Variable
8. Pointers and Arrays
9. Pointer Arithmetic
10. Use of Unary Operators (++, --)
11. One-Dimensional Arrays and Pointer
12. Concept of Array of Pointers and Simple Use
13. Command Line Arguments for the main() Function
14. Comparison Section
15. Programming Examples
16. Quick Revision

TOPIC 01

Concept of Pointers

1.1 Definition

A **pointer** is a special variable that does not store an ordinary data value (like a number or a character) but instead stores the **memory address** of another variable. In other words, a pointer "points to" the location in memory where a value is stored, rather than storing the value itself.

> **Formal Definition:** A pointer is a derived data type in C that holds the address of another variable of a specific data type.

1.2 Need for Pointers

Pointers are one of the most powerful features of the C language. They are required because of the following reasons:

1. Direct Access and Manipulation of Memory

They allow **direct access and manipulation** of memory locations, giving low-level control similar to assembly language.

```
int a = 10;
int *p = &a;
*p = 99;              /* directly modifies the memory location of a */
printf("%d", a);      /* prints 99 */
```

```
Output
99
```

2. Call by Reference

They enable **call by reference**, so a function can modify the actual arguments passed to it (normal function calls in C are call by value only).

```
void addTen(int x)       { x = x + 10; }      /* call by value: no effect outside   */
void addTenRef(int *x)   { *x = *x + 10; }     /* call by reference: changes caller  */

int main() {
    int n = 5;
    addTen(n);         printf("%d\n", n);     /* prints 5  (unchanged) */
    addTenRef(&n);     printf("%d\n", n);     /* prints 15 (changed)   */
}
```

```
Output
5
15
```

3. Dynamic Memory Allocation

They allow **dynamic memory allocation** (malloc, calloc, realloc, free), so memory can be allocated and released while the program is running.

```
int n;
scanf("%d", &n);
int *arr = (int *) malloc(n * sizeof(int));   /* memory created at runtime */
for (int i = 0; i < n; i++) arr[i] = i + 1;
free(arr);                                     /* memory released when done */
```

4. Dynamic Data Structures

They make it possible to build and traverse **dynamic data structures** such as linked lists, stacks, queues, trees, and graphs.

```
struct Node {
    int data;
    struct Node *next;   /* pointer links this node to the next one */
};
```

5. Arrays and Strings

They provide an efficient way to work with **arrays and strings**, since array names decay into pointers.

```
int arr[3] = {10, 20, 30};
printf("%d", *(arr + 1));   /* prints 20, using pointer notation on the array */
```

```
Output
20
```

6. Returning Multiple Values

They allow a program to return **more than one value** from a function (through pointer parameters).

```
void minMax(int a, int b, int *min, int *max) {
    if (a < b) { *min = a; *max = b; }
    else       { *min = b; *max = a; }
}
int main() {
    int lo, hi;
    minMax(15, 7, &lo, &hi);
    printf("min=%d max=%d\n", lo, hi);
}
```

```
Output
min=7 max=15
```

7. Function Pointers

They support **function pointers**, which allow functions to be passed as arguments and enable callback-style programming.

```
int square(int x) { return x * x; }
int main() {
    int (*fp)(int) = square;   /* fp points to function square */
    printf("%d", fp(5));        /* calls square(5) through fp   */
}
```

```
Output
25
```

1.3 Advantages and Applications

Advantages

- Efficient memory usage
- Faster execution (direct address access)
- Enables dynamic memory management
- Makes call by reference possible
- Simplifies handling of arrays, strings, and structures

Applications

- Dynamic data structures (linked list, tree, graph)
- Passing large structures to functions efficiently
- Operating system memory management
- Implementing arrays of variable size at runtime
- Function callbacks and pointer-to-function based APIs

1.4 Diagram: Pointer Concept in Memory

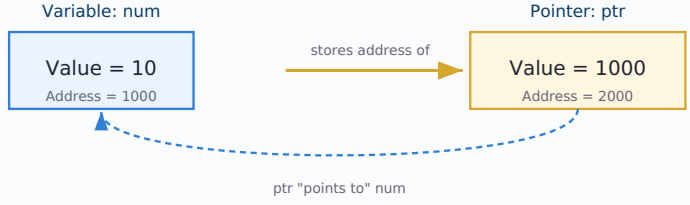


Fig 1.1 — The pointer variable ptr (at address 2000) stores the value 1000, which is the address of variable num.

Exam Tip: A very common one-line question is "What does a pointer contain?" — Answer: the address of another variable, not the value itself.

Chapter Summary — Concept of Pointers

- A pointer stores the address of a variable, not its value.
- Needed for call by reference, dynamic memory allocation, and efficient data structures.
- Main advantages: memory efficiency, speed, and flexibility in handling arrays/structures.

TOPIC 02

Pointer as an Address Variable

2.1 Definition

Since a pointer's only job is to hold an address, it is often called an **address variable**. Every variable created in a C program is stored somewhere in the computer's memory (RAM), and that location is identified by a unique **memory address**, usually shown in hexadecimal form (e.g., 0x7ffee23a).

2.2 Memory Addresses

- Every byte of computer memory has a unique address.
- When a variable is declared, the compiler allocates a block of memory (size depends on data type) and associates it with the variable name.
- The address of this block can be obtained using the **&** (address) operator.

2.3 Declaring Pointer Variables

Declaring a pointer variable is the process of telling the compiler that a variable will store the address of another variable of a specific data type, rather than an ordinary value.

SYNTAX

```
datatype *pointer_name;
```

```
int  *p;      /* pointer to int    */
float *fp;    /* pointer to float  */
char  *cp;    /* pointer to char   */
```

Note: The asterisk (*) tells the compiler that the variable being declared is a pointer, and the datatype tells it what type of variable's address it is meant to hold.

2.4 Examples with Memory Representation

```
#include <stdio.h>
int main() {
    int num = 25;
    int *p;
    p = &num;
    printf("Value of num  : %d\n", num);
    printf("Address of num: %p\n", &num);
    printf("Value of p    : %p\n", p);
    return 0;
}
```

Output

```
Value of num  : 25
Address of num: 0x7ffe3a1c
Value of p    : 0x7ffe3a1c
```

Address	Variable	Value
0x7ffe3a1c	num	25
0x7ffe3a20	p	0x7ffe3a1c

Fig 2.1 — Pointer p (at address 0x7ffe3a20) holds the address of num (0x7ffe3a1c).

2.5 Advantages, Limitations & Common Errors

Advantages

Limitations

Advantages

- Gives a name/handle to a raw memory address, making it usable in a type-safe way.
- Forms the base for every other pointer operation (dereferencing, arithmetic).

Limitations

- An address by itself is meaningless without knowing the type of data stored there.
- Addresses are machine/run dependent — cannot be hard-coded or assumed constant.

Common Mistake: Assigning an address of one type to a pointer of a mismatched type without a cast (e.g., `int *p = &floatVar;`) — the compiler issues a warning/error because the two types are incompatible.

Chapter Summary — Pointer as an Address Variable

- A pointer is essentially an "address variable."
- Declared using the form *datatype *name;*
- Its own value is always an address, printed conventionally using %p.

TOPIC 03

Concept of a Pointer Data Type and its Syntax

3.1 Pointer Declaration

A pointer must always be declared with the data type of the variable it will point to. This is because the compiler needs to know how many bytes to read/write when the pointer is dereferenced, and how much to add/subtract during pointer arithmetic.

GENERAL FORM

```
datatype *pointer_name;
```

3.2 Initialization

Initializing a pointer means giving it the address of a variable to point to. Before this is done, the pointer holds a garbage address and should not be used.

```
int a = 10;
int *p = &a;      /* declaration + initialization */

int *q;             /* declaration only        */
q = &a;             /* initialization separately */
```

Common Mistake: An uninitialized pointer is called a **wild pointer**. Using it (dereferencing) leads to undefined behavior or a program crash. Always initialize a pointer to NULL if its target is not yet known: `int *p = NULL;`

3.3 Pointer Data Types

C allows a pointer to be created for every fundamental and derived data type. Although all pointers store an address (and hence typically occupy the same number of bytes on a given system — commonly 8 bytes on 64-bit systems), their **type** determines how the data at that address is interpreted.

Pointer Type	Declaration	Points To
Integer pointer	int *p;	int variable (commonly 4 bytes)
Character pointer	char *p;	char variable (1 byte)
Float pointer	float *p;	float variable (commonly 4 bytes)
Double pointer (value)	double *p;	double variable (commonly 8 bytes)
Void pointer	void *p;	Generic — any data type
Pointer to pointer	int **p;	Another pointer variable

3.4 Syntax and Examples

```
int    a = 5;    int    *pa = &a;
float  b = 5.5;  float  *pb = &b;
char   c = 'A';  char   *pc = &c;
double d = 5.55; double *pd = &d;
```

Exam Tip: "sizeof(any_pointer)" gives the same result on a given machine (e.g., 8 bytes on a 64-bit system) regardless of the datatype it points to — because a pointer always just stores an address. Only **void*** cannot be dereferenced directly without a cast.

3.5 Advantages, Limitations & Common Errors

Advantages

- Lets the compiler check types and calculate pointer arithmetic correctly.
- NULL initialization provides a safe, checkable "points to nothing" state.

Limitations

- An untyped/void pointer cannot be dereferenced without an explicit cast.
- Declaring a pointer does not allocate memory for the data it will point to.

Common Mistake: Assuming `int *p;` allocates space for an integer — it only allocates space for the pointer itself; the target memory must already exist or be allocated separately (e.g., via malloc).

Chapter Summary — Pointer Data Type and Syntax

- General syntax: *datatype *pointer_name;*
- Pointer type decides how many bytes are read at the address it holds.
- Uninitialized pointers (wild pointers) are dangerous; initialize to NULL if unused.

TOPIC 04

Built-in Address Operator (&)

4.1 Purpose

The **address operator (&)** is a unary operator in C that, when applied to a variable, returns the memory address at which that variable is stored. It is the fundamental operator used to obtain the value that will be stored inside a pointer.

4.2 Working Mechanism

- When the compiler sees `&variable`, it does not access the value of the variable; instead it looks up the memory location assigned to that variable.
- This address (a numeric value, shown in hexadecimal) can be stored only in a pointer variable of a matching type.
- The address operator can be applied to any variable — but not to constants, expressions, or register-class variables.

4.3 Syntax

SYNTAX

```
&variable_name
```

4.4 Examples with Output

```c
#include <stdio.h>
int main() {
    int x = 15;
    printf("Value of x   : %d\n", x);
    printf("Address of x : %p\n", &x);
    return 0;
}
```

Output

```
Value of x   : 15
Address of x : 0x7ffd914ac
(Address values change every run/machine)
```

Note: The exact hexadecimal address printed differs across systems and even across different runs of the program (due to OS-level memory allocation/ASLR); only the concept is examined, not a fixed numeric value.

4.5 Diagram — Address Operator (&) Working

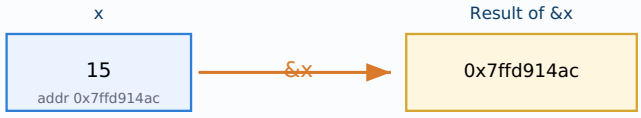


Fig 4.1 — Applying & to variable x returns its memory address, which can then be stored in a pointer.

4.6 Advantages, Limitations & Common Errors

Advantages

- Only way to obtain a variable's address for use in pointers/call by reference.
- Lightweight — resolved at compile time, no runtime overhead.

Limitations

- Cannot be applied to constants, literals, or expressions (e.g., &(a+b) is invalid).
- Cannot be applied to variables declared with the register storage class (no guaranteed memory address).

Common Mistake: Writing &5 or &(a+b) — the address operator can only be applied to an lvalue (a named, addressable variable), not to a constant or the result of an expression.

Chapter Summary — Address Operator (&)

- & returns the address of a variable.
- Essential for initializing/assigning pointers.
- Cannot be applied to constants or expressions.

TOPIC 05

Pointers to Existing Variables of Different Data Types and Their Uses

5.1 Integer Pointers

```
int a = 100;
int *pi = &a;
printf("%d", *pi);   /* prints 100 */
```

An integer pointer moves in steps of **4 bytes** (typical int size) when incremented, and interprets the memory at its address as a whole number.

Uses: Traversing/indexing integer arrays, counters and loop variables passed by reference, and building integer-based data structures (linked lists, trees).

5.2 Character Pointers

```
char ch = 'Z';
char *pc = &ch;
printf("%c", *pc);   /* prints Z */
```

A character pointer moves in steps of **1 byte**. Character pointers are also widely used to represent and traverse strings, since a string is stored as an array of characters terminated by '\0'.

Uses: Representing and manipulating strings, reading input character-by-character, and implementing functions like string copy/compare/length.

5.3 Float Pointers

```
float f = 12.5;
float *pf = &f;
printf("%f", *pf);   /* prints 12.500000 */
```

A float pointer moves in steps of **4 bytes** and interprets the bytes at the address using IEEE-754 floating point representation.

Uses: Handling real-number data such as measurements, coordinates, and scientific calculations passed to functions by reference.

What is IEEE-754? IEEE-754 is the standard that defines how computers store decimal (real) numbers like 12.5 or 3.14 in binary — it's what float and double actually use under the hood in C.

5.4 Double Pointers (Pointer to Double)

```
double d = 99.99;
double *pd = &d;
printf("%lf", *pd);  /* prints 99.990000 */
```

A pointer to double moves in steps of **8 bytes**. (Note: "double pointer" here means a pointer to a *double* variable — not to be confused with a "pointer to pointer", which is discussed under Array of Pointers/general pointer theory.)

Uses: High-precision computations such as financial and scientific data, where extra decimal precision beyond float is required.

5.5 Memory Diagram — Different Pointer Types

Variable	Type	Address	Bytes Occupied	Pointer
a = 100	int	3000	4	int *pi = 3000
ch = 'Z'	char	3010	1	char *pc = 3010

f = 12.5	float	3020	4	float *pf = 3020
d = 99.99	double	3030	8	double *pd = 3030

Fig 5.1 — Each pointer type stores an address, but "knows" the size/interpretation of data at that address.

5.6 Applications

- Integer pointers — array traversal, indexing, counters.
- Character pointers — string handling, dynamic strings, command-line arguments.
- Float/double pointers — scientific/financial computation buffers, passing large real numbers to functions efficiently.

Exam Tip: A frequently asked MCQ: "Why must a pointer be declared with a specific data type?" — Answer: so that pointer arithmetic and dereferencing correctly compute the number of bytes to move/read.

5.7 Advantages, Limitations & Common Errors

Advantages

- Typed pointers allow the compiler to catch type-mismatch errors at compile time.
- Correct step size in pointer arithmetic is automatically calculated using the type.

Limitations

- A pointer of one type cannot directly access data of another type without an explicit cast.
- Mixing pointer types (e.g., assigning float* to int*) causes data misinterpretation.

Common Mistake: Using a char pointer to read/print an int variable's address (or vice-versa) — this reads the wrong number of bytes and prints garbage/incorrect values.

Chapter Summary — Pointers to Different Data Types

- int*, char*, float*, double* each move by their respective type size during arithmetic.
- The type of a pointer determines how the bytes at its address are interpreted.

TOPIC 06

Use of Indirection Operator (*)

6.1 Dereferencing Pointers

The **indirection operator** (also called the **dereference operator**), written as *, is used to access or modify the value stored at the address held by a pointer. Applying * to a pointer "goes to" the address it stores and retrieves/sets the value there.

SYNTAX

```
*pointer_name        /* access value pointed to */
```

6.2 Reading Values Through Pointers

```
int a = 50;
int *p = &a;
printf("%d", *p);   /* reads value of a through p -> 50 */
```

6.3 Modifying Values Through Pointers

```
#include <stdio.h>
int main() {
    int a = 50;
    int *p = &a;
    *p = 75;              /* changes a indirectly */
    printf("a = %d", a);
    return 0;
}
```

Output

```
a = 75
```

Here, `*p = 75;` does not change the pointer p — it changes the value stored at the address p points to, i.e., the value of a.

6.4 Memory Representation

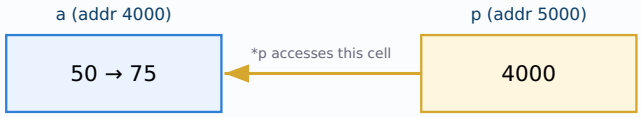


*Fig 6.1 — *p reads/writes the memory cell at address 4000 (i.e., variable a), changing its value from 50 to 75.*

Common Mistake: Confusing * used in declaration (`int *p;` — meaning "p is a pointer") with * used in an expression (*p — meaning "value at address p"). The same symbol has two different meanings depending on context.

6.5 Advantages, Limitations & Common Errors

Advantages

- Allows a function to modify a caller's variable directly (call by reference).
- Enables generic, reusable access to data through a single pointer variable.

Limitations

- Dereferencing a NULL or uninitialized pointer crashes the program (segmentation fault).
- Adds a small performance/readability overhead of indirection versus direct variable access.

Chapter Summary — Indirection Operator (*)

- *p accesses/modifies the value at the address stored in p.

- Also called dereferencing.
- Meaning of * differs in declaration vs expression context.

TOPIC 07

The Name of the Array as a Pointer Variable

7.1 Relationship Between Array Name and Pointer

In C, the name of an array, when used in most expressions, automatically **decays** into a pointer to its first element. That is, for an array `int arr[5]`, the expression `arr` is equivalent in value to `&arr[0]`.

Important Distinction: The array name is not itself a pointer variable — it is a constant address (cannot be reassigned) — but it behaves like a pointer to the first element in expressions.

7.2 Array Base Address

```
RELATION
arr  ==  &arr[0]   (base address of the array)
```

7.3 Examples

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    printf("arr       = %p\n", arr);
    printf("&arr[0]   = %p\n", &arr[0]);
    printf("*arr      = %d\n", *arr);
    return 0;
}
```

```
Output
arr       = 0x7ffe1000
&arr[0]   = 0x7ffe1000
*arr      = 10
```

7.4 Memory Layout Diagram

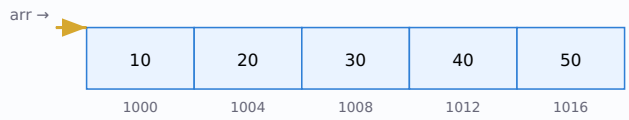


Fig 7.1 — arr points to the first element (address 1000); consecutive int elements are 4 bytes apart.

Exam Tip: "arr" gives base address; "&arr" gives address of the whole array (same numeric value but type *int (*)[5]*, different pointer arithmetic behaviour). This distinction is a favourite exam trap.

7.5 Advantages, Limitations & Common Errors

Advantages

- Lets array elements be accessed and manipulated using fast pointer arithmetic.
- Enables arrays to be passed to functions efficiently as just a base address.

Limitations

- Because arr is a constant, it cannot be reassigned to point to a different array.
- The decayed pointer loses the original array's size information.

Common Mistake: Writing `arr = arr2;` to try to make one array name refer to another array — illegal in C since the array name is a non-modifiable (constant) address.

Chapter Summary — Array Name as Pointer

- Array name decays to pointer to first element in expressions.
- arr is a constant pointer; it cannot be made to point elsewhere.
- arr and &arr have same address value but different types.

TOPIC 08

Pointers and Arrays

8.1 Accessing Array Elements Using Pointers

Since an array name decays into a pointer to its first element, any element of the array can also be reached by adding an offset to a pointer and dereferencing it, instead of using index (square-bracket) notation.

```
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = arr;
    for (int i = 0; i < 5; i++)
        printf("%d ", *(p + i));
    return 0;
}
```

```
Output
10 20 30 40 50
```

8.2 Pointer Notation vs Array Notation

C guarantees that `arr[i]` and `*(arr + i)` always give the same result, because array indexing is internally defined in terms of pointer arithmetic.

Array Notation	Equivalent Pointer Notation	Meaning
arr[0]	*(arr + 0) or *p	First element
arr[i]	*(arr + i) or *(p + i)	i-th element
&arr[i]	arr + i or p + i	Address of i-th element

8.3 Examples

```
int arr[3] = {1, 2, 3};
int *p = arr;
printf("%d %d\n", arr[1], *(p+1));   /* both print 2 */
printf("%p %p\n", &arr[1], p+1);     /* same address */
```

```
Output
2 2
0x7ffe1004 0x7ffe1004   (address will vary)
```

8.4 Comparison Table

Aspect	Array	Pointer
Memory allocation	Fixed at compile time (static)	Can point to dynamically allocated memory
Reassignment	Array name cannot be reassigned	Pointer variable can be made to point elsewhere
sizeof	Gives total size of array (bytes)	Gives size of the pointer itself (e.g., 8 bytes)
Initialization	Elements initialized individually / with list	Initialized with an address

Common Mistake: Believing arrays and pointers are "the same thing." They behave similarly in expressions but `sizeof(arr)` and `sizeof(p)` give very different results — this is a classic exam question.

8.5 Advantages, Limitations & Common Errors

Advantages

- Pointer notation allows flexible, dynamic traversal (e.g., using a moving pointer variable).

Limitations

- Pointer notation is generally less readable/self-documenting than indexed array notation.

- Interchangeability with array notation gives programmers a choice of the clearer style for a task.
- Unlike arr[i], a plain pointer p does not carry any built-in bounds checking.

Chapter Summary — Pointers and Arrays

- arr[i] and *(arr + i) are equivalent.
- Pointer notation and array notation are interchangeable for element access, but arrays and pointers are not identical in memory behaviour.

TOPIC 09

Pointer Arithmetic

Pointer arithmetic refers to the set of mathematical operations — increment, decrement, addition, subtraction, and comparison — that can be performed on pointer variables. Unlike ordinary arithmetic, these operations are automatically scaled by the size of the data type the pointer points to, which is what allows a pointer to move cleanly from one array element to the next.

9.1 Increment and Decrement (++, --)

When a pointer is incremented (p++) or decremented (p--), it does not move by 1 byte — it moves by **sizeof(type)** bytes, so that it lands exactly on the next/previous element of that type.

```
#include <stdio.h>
int main() {
    int arr[3] = {10, 20, 30};
    int *p = arr;
    printf("%d\n", *p);   /* p -> arr[0] */
    p++;
    printf("%d\n", *p);   /* p -> arr[1] */
    p--;
    printf("%d\n", *p);   /* back to arr[0] */
    return 0;
}
```

```
Output
10
20
10
```

9.2 Addition and Subtraction

Besides ++ and --, a pointer can be moved forward or backward by any whole number of elements in a single step by adding or subtracting an integer, and two pointers into the same array can be subtracted to find how many elements lie between them.

```
RULES
p + n   moves forward by  n * sizeof(type) bytes
p - n   moves backward by n * sizeof(type) bytes
p2 - p1 gives the number of elements between two pointers (not bytes)
```

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int *p = &arr[0];
    int *q = &arr[4];

    printf("*(p+2) = %d\n", *(p + 2));   /* jump forward 2 elements */
    printf("*(q-1) = %d\n", *(q - 1));   /* jump backward 1 element */
    printf("q - p  = %ld\n", q - p);     /* number of elements apart */
    return 0;
}
```

```
Output
*(p+2) = 3
*(q-1) = 4
q - p  = 4
```

9.3 Pointer Comparison

Two pointers pointing into the **same array** can be validly compared using `<`, `>`, `==`, `!=` to determine relative position. Comparing pointers to unrelated memory blocks is not meaningful/defined.

```
#include <stdio.h>
int main() {
    int arr[4] = {5, 10, 15, 20};
    int *p = &arr[0];
```

```
    int *q = &arr[3];

    if (p < q)
        printf("p is before q in memory\n");
    if (p != q)
        printf("p and q point to different elements\n");
    return 0;
}
```

```
Output
p is before q in memory
p and q point to different elements
```

9.4 Rules of Pointer Arithmetic

- Only addition/subtraction of an **integer** to/from a pointer is allowed (not pointer + pointer).
- Subtracting two pointers of the **same type** gives the number of elements between them.
- Multiplication or division of pointers is **not allowed**.
- Pointer arithmetic is only well-defined within the bounds of the same array (plus one position past the end).
- A void pointer cannot be used in arithmetic directly (size unknown) in standard C.

```
int arr[3] = {1, 2, 3};
int *p = &arr[0], *q = &arr[2];

int  x  = *(p + 1);   /* VALID   -> integer added to pointer, x = 2       */
long n  = q - p;       /* VALID   -> pointer - pointer (same type), n = 2 */
/* int  y = p + q; */  /* INVALID -> pointer + pointer is illegal          */
/* int  z = p * 2; */  /* INVALID -> pointer * integer is illegal          */
```

9.5 Memory Diagram — Pointer Arithmetic

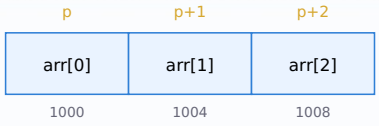


*Fig 9.1 — For int *p, each "+1" step moves exactly 4 bytes to land on the next array element.*

Exam Tip: If a char pointer and int pointer both start at the same address and are incremented once, the char pointer moves 1 byte ahead while the int pointer moves 4 bytes ahead — a favourite viva/short-answer question.

9.6 Advantages, Limitations & Common Errors

Advantages

- Enables fast, direct traversal of arrays without repeated index computation.
- Forms the basis of efficient string and buffer processing.

Limitations

- No bounds checking — moving a pointer past an array's limits is undefined behaviour.
- Not allowed on void pointers directly, since element size is unknown.

Common Mistake: Advancing a pointer beyond the last valid element (or before the first) and then dereferencing it — this reads/writes unrelated memory and can crash the program or corrupt data.

Chapter Summary — Pointer Arithmetic

- p + n moves n*sizeof(type) bytes; p - n moves back similarly.
- q - p (same type) gives number of elements apart.
- Multiplication/division of pointers is illegal.

TOPIC 10

Use of Unary Operators (++, --)

10.1 Pre-increment and Post-increment on Pointers

Just like with ordinary variables, a pointer can be incremented either before it is used in an expression (pre-increment, ++p) or after (post-increment, p++) — the difference lies in whether the pointer moves forward before or after its current value is dereferenced.

```
int arr[3] = {10, 20, 30};
int *p = arr;

printf("%d\n", *p++);   /* post-inc: uses *p (10), THEN p++     */
printf("%d\n", *p);     /* now p -> arr[1], prints 20            */

int *q = arr;
printf("%d\n", *++q);   /* pre-inc: q++ FIRST, then dereference  */
```

```
Output
10
20
20
```

Uses: Post-increment (p++) is the standard idiom for stepping through array elements or strings one at a time in a loop; pre-increment (++p) is preferred when the pointer must be advanced first and its new position used immediately, such as skipping a leading element before processing.

Usage Example — Traversing a String with Post-increment:

```
#include <stdio.h>
int main() {
    char str[] = "CODE";
    char *p = str;
    while (*p != '\0') {
        printf("%c ", *p++);   /* print current char, then advance */
    }
    return 0;
}
```

```
Output
C O D E
```

Usage Example — Skipping the First Element with Pre-increment:

```
#include <stdio.h>
int main() {
    int marks[4] = {100, 70, 80, 90};   /* marks[0] is a placeholder/roll-no slot */
    int *p = marks;
    printf("%d\n", *++p);   /* skip marks[0], read marks[1] directly = 70 */
    return 0;
}
```

```
Output
70
```

10.2 Pre-decrement and Post-decrement

Decrementing a pointer works the same way in reverse: pre-decrement (`--p`) moves the pointer backward before dereferencing, while post-decrement (`p--`) dereferences the current value first and then moves the pointer backward.

```
int arr[3] = {10, 20, 30};
int *p = &arr[2];
printf("%d\n", *p--);   /* uses arr[2] then moves p back  */
printf("%d\n", *p);     /* now points to arr[1]           */
```

```
Output
```

```
30
20
```

Uses: Commonly used to traverse an array or string backward (e.g., from the last element to the first), such as reversing a string in place or scanning a buffer from its end.

Usage Example — Printing an Array in Reverse with Post-decrement:

```c
#include <stdio.h>
int main() {
    int arr[5] = {10, 20, 30, 40, 50};
    int *p = &arr[4];         /* start at the last element */
    for (int i = 0; i < 5; i++)
        printf("%d ", *p--);   /* print current, then move backward */
    return 0;
}
```

```
Output
50 40 30 20 10
```

10.3 Working Mechanism (Operator Precedence)

Expression	Meaning
*p++	Equivalent to *(p++) — dereference current value, then advance p (unary * and postfix ++ — postfix binds first, but returns old p for dereference)
*++p	Equivalent to *(++p) — advance p first, then dereference
(*p)++	Increment the value pointed to by p (not the pointer itself)
++*p	Same as (*p)++ in effect — increments the pointed-to value first

Common Mistake: Confusing *p++ (moves the pointer) with (*p)++ (changes the value at that address). Since * and postfix ++ have the same precedence but postfix ++ has higher priority and associates left-to-right with the pointer, *p++ is parsed as *(p++).

10.4 Advantages, Limitations & Common Errors

Advantages

- Compact syntax lets a pointer be advanced and its old/new value used in a single expression.
- Very efficient for tight loops such as array/string traversal.

Limitations

- Precedence rules make combined expressions like *p++ error-prone for beginners.
- Overuse can hurt code readability and make debugging harder.

Chapter Summary — Unary Operators on Pointers

- *p++ dereferences first (old address), then moves pointer forward.
- *++p moves pointer forward first, then dereferences.
- (*p)++ changes the value being pointed to, not the pointer.

TOPIC 11

One-Dimensional Arrays and Pointer

A **one-dimensional (1-D) array** stores its elements in one continuous, contiguous block of memory. Because the elements are laid out back-to-back with no gaps, a single pointer can walk through the entire array simply by being advanced one step at a time — this is what makes pointers and 1-D arrays so closely linked in C.

11.1 Traversing Arrays Using Pointers

```c
#include <stdio.h>
int main() {
    int arr[5] = {2, 4, 6, 8, 10};
    int *p = arr;
    for (int i = 0; i < 5; i++)
        printf("%d ", *(p + i));
    return 0;
}
```

```
Output
2 4 6 8 10
```

11.2 Accessing Elements

Any element of a 1-D array can be accessed via a pointer using either `*(p+i)` or, once the pointer is advanced, simply `*p` repeatedly with p++ in a loop.

```c
#include <stdio.h>
int main() {
    int arr[5] = {2, 4, 6, 8, 10};
    int *p = arr;
    while (p < arr + 5) {
        printf("%d ", *p);
        p++;
    }
    return 0;
}
```

```
Output
2 4 6 8 10
```

11.3 Passing Arrays to Functions

In C, arrays are always passed to functions **by reference** (i.e., the base address is passed), because the array name decays into a pointer. Any changes made inside the function reflect in the original array.

```
SYNTAX
void func(int *p, int n);        // or
void func(int p[], int n);       // equivalent
```

```c
#include <stdio.h>
void doubleValues(int *p, int n) {
    for (int i = 0; i < n; i++)
        p[i] = p[i] * 2;
}
int main() {
    int arr[4] = {1, 2, 3, 4};
    doubleValues(arr, 4);
    for (int i = 0; i < 4; i++)
        printf("%d ", arr[i]);
    return 0;
}
```

```
Output
2 4 6 8
```

Note: Since the array's base address is passed, `sizeof(p)` inside the function gives the size of a pointer (e.g., 8), not the size of the whole array — the array "decays" once passed as a parameter.

11.4 Memory Layout Diagram

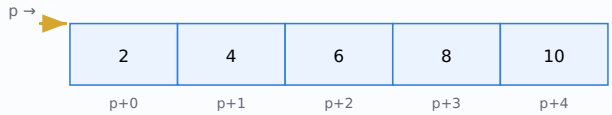


Fig 11.1 — A single pointer p walks across the whole 1-D array, one contiguous int-sized step at a time.

Exam Tip: A common short-answer question asks to rewrite an index-based array loop using pointer notation (and vice versa) — remember `arr[i]` and `*(p+i)` always produce the same result for a 1-D array.

11.5 Advantages, Limitations & Common Errors

Advantages

- Pointer-based traversal is often faster than index-based access (no repeated base+offset computation by some compilers).
- Allows functions to modify the caller's array directly without returning a new array.

Limitations

- Array size information is lost once passed to a function — must be passed separately.
- Pointer-based loops can be harder to read/debug than clear index-based loops.

Common Mistake: Trying to compute array length inside a function using `sizeof(arr)/sizeof(arr[0])` — this fails because arr has already decayed to a pointer; always pass the length as a separate parameter.

Chapter Summary — 1-D Arrays and Pointers

- Pointers traverse arrays using *(p+i) or p++ in a loop.
- Arrays passed to functions decay to pointers — changes reflect in the caller.

TOPIC 12

Concept of Array of Pointers and Simple Use

12.1 Definition

An **array of pointers** is an array in which each element is itself a pointer variable, rather than an ordinary data value. It is especially useful for handling arrays of strings (since a string in C is itself a char pointer or char array).

12.2 Declaration and Initialization

SYNTAX

```
datatype *arr_name[size];
```

```
char *names[3] = {"Amit", "Sneha", "Ravi"};
int a = 1, b = 2, c = 3;
int *iarr[3] = {&a, &b, &c};
```

12.3 Accessing Elements

```
#include <stdio.h>
int main() {
    char *names[3] = {"Amit", "Sneha", "Ravi"};
    for (int i = 0; i < 3; i++)
        printf("%s\n", names[i]);
    return 0;
}
```

Output

```
Amit
Sneha
Ravi
```

12.4 Applications

- Storing and managing a list of strings (e.g., menu options, names, words).
- Implementing `argv[]` in command-line arguments (see Topic 13).
- Building simple lookup tables or jump tables using function pointer arrays.

12.5 Memory Representation

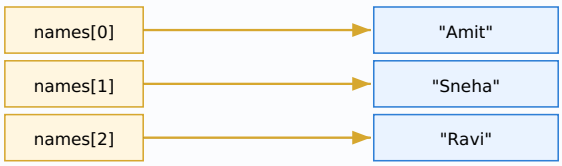


Fig 12.1 — Each element of the pointer array stores the address of a separate string stored elsewhere in memory.

Exam Tip: Distinguish **array of pointers** (`char *names[3]`) from a true **2-D character array** (`char names[3][10]`) — the former stores addresses of strings of varying length; the latter reserves fixed, equal-sized rows.

12.6 Advantages, Limitations & Common Errors

Advantages

- Saves memory when strings have unequal lengths (no wasted padding, unlike a fixed 2-D array).
- Easy to rearrange/sort by simply swapping pointers rather than

Limitations

- Each pointer must be validly initialized; uninitialized elements are wild pointers.
- Extra indirection (one more memory access) compared to a flat

copying whole strings.

2-D array.

Common Mistake: Declaring `char *names[3];` without initializing each pointer to point to actual string storage, then trying to write into it directly — this causes undefined behaviour since no memory has been allocated for the string data.

Chapter Summary — Array of Pointers

- Declared as *datatype *arr[size];* — each element is a pointer.
- Commonly used for arrays of strings and for argv[] in main().

TOPIC 13

Command Line Arguments for the main() Function

13.1 Definition

Command line arguments are values passed to a C program from the operating system's command prompt/terminal at the time of program execution. They allow a program to receive input without using scanf, directly when it is launched.

13.2 Syntax (argc, argv)

SYNTAX

```
int main(int argc, char *argv[]) {
    ...
}
```

Parameter	Meaning
argc	"Argument Count" — number of command-line arguments (including program name)
argv[]	"Argument Vector" — array of pointers to strings, each string is one argument; argv[0] is the program name
argv[argc]	Always NULL (marks end of the array)

13.3 Working Mechanism

- When the program is run as `./program hello 123`, the OS splits input by spaces and builds an array of strings.
- argc becomes 3 (program name + 2 arguments); argv[0]="./program", argv[1]="hello", argv[2]="123".
- All arguments are received as strings (char*) — numeric arguments must be converted using functions like atoi()/atof() if needed.

13.4 Passing Arguments — Practical Example

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    printf("Number of arguments: %d\n", argc);
    for (int i = 0; i < argc; i++)
        printf("argv[%d] = %s\n", i, argv[i]);
    return 0;
}
```

13.5 Program Execution Example

```
$ gcc program.c -o program
$ ./program Hello World 2026
```

Output

```
Number of arguments: 4
argv[0] = ./program
argv[1] = Hello
argv[2] = World
argv[3] = 2026
```

13.6 Diagram — argc / argv Structure



Fig 13.1 — argv is an array of pointers to strings; argv[argc] is always NULL.

Exam Tip: argv[0] is always the program's own name/path — a very common one-mark question.

13.7 Advantages, Limitations & Common Errors

Advantages

- Allows configuring a program's behaviour at launch without modifying source code.
- Widely used for scripting, automation, and passing file names/options to utilities.

Limitations

- All arguments arrive only as strings — explicit conversion is needed for numeric use.
- No built-in validation; the program must check argc and argument format itself.

Common Mistake: Accessing `argv[i]` without first checking that `argc` is large enough — this reads past the valid arguments and leads to undefined behaviour.

Chapter Summary — Command Line Arguments

- int main(int argc, char *argv[]) receives command-line input.
- argc = count of arguments; argv = array of pointers to argument strings.
- All arguments arrive as strings; convert manually if numeric values are needed.

TOPIC 14

Comparison Section

14.1 Pointer vs Variable

Aspect	Ordinary Variable	Pointer Variable
Stores	An actual data value	The address of another variable
Access	Directly used by name	Requires dereferencing (*) to get value
Size	Depends on data type	Fixed for a given system (e.g., 8 bytes on 64-bit)
Usage	Simple computations	Dynamic memory, call by reference, data structures

14.2 Array vs Pointer

Aspect	Array	Pointer
Memory	Contiguous block, fixed at declaration	Can be reassigned to point anywhere, including dynamic memory
Name reassignment	Not allowed (constant address)	Allowed
sizeof	Total array size in bytes	Size of the pointer variable itself
Initialization requirement	Elements can remain uninitialized	Should be initialized before use (else wild pointer)

14.3 Pointer Notation vs Array Notation

Array Notation	Pointer Notation
arr[i]	*(arr + i)
&arr[i]	arr + i
arr[0]	*arr

14.4 Address Operator (&) vs Indirection Operator (*)

Aspect	Address Operator (&)	Indirection Operator (*)
Also called	Reference operator	Dereference operator
Applied to	An ordinary variable	A pointer variable
Returns	The address of the variable	The value stored at that address
Example	&a gives address of a	*p gives value pointed to by p
Nature	Unary operator	Unary operator

14.5 Comprehensive Comparison Table

Concept	Key Point
Pointer	Stores address of another variable
Array	Contiguous, fixed-size collection with a constant base address
& operator	Gets address of a variable
* operator	Gets/sets value at an address (dereferencing)
Pointer arithmetic	Scaled by sizeof(type)
Array of pointers	Each element is itself an address
argc/argv	Mechanism to pass command-line input into main()

Chapter Summary — Comparisons

- Pointer ≠ variable: pointer stores address, variable stores value.
- Array name behaves like a pointer but is a constant address, not reassignable.
- & fetches an address; * fetches/modifies the value at an address.

TOPIC 15

Programming Examples

15.1 Basic Pointer Program

```
#include <stdio.h>
int main() {
    int a = 20;
    int *p = &a;
    printf("Value  : %d\n", *p);
    printf("Address: %p\n", p);
    return 0;
}
```

```
Output
Value  : 20
Address: 0x7ffe2a10   (address will vary)
```

Explanation: p stores the address of a; *p dereferences p to fetch a's value.

15.2 Swapping Two Numbers Using Pointers (Call by Reference)

```
#include <stdio.h>
void swap(int *x, int *y) {
    int temp = *x;
    *x = *y;
    *y = temp;
}
int main() {
    int a = 5, b = 10;
    swap(&a, &b);
    printf("a = %d, b = %d\n", a, b);
    return 0;
}
```

```
Output
a = 10, b = 5
```

Explanation: Addresses of a and b are passed, so swap() modifies the original variables directly — this is true call by reference using pointers.

15.3 Pointer Arithmetic Program

```
#include <stdio.h>
int main() {
    int arr[4] = {5, 10, 15, 20};
    int *p = arr;
    for (int i = 0; i < 4; i++) {
        printf("*(p+%d) = %d\n", i, *(p + i));
    }
    return 0;
}
```

```
Output
*(p+0) = 5
*(p+1) = 10
*(p+2) = 15
*(p+3) = 20
```

15.4 Array Traversal Using Pointers (Sum of Elements)

```
#include <stdio.h>
int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int *p = arr, sum = 0;
    for (int i = 0; i < 5; i++)
        sum += *(p + i);
    printf("Sum = %d\n", sum);
```

```
    return 0;
}
```

```
Output
Sum = 15
```

15.5 Array of Pointers Example

```
#include <stdio.h>
int main() {
    char *fruits[3] = {"Apple", "Mango", "Banana"};
    for (int i = 0; i < 3; i++)
        printf("%s\n", fruits[i]);
    return 0;
}
```

```
Output
Apple
Mango
Banana
```

15.6 Command-Line Argument Program

```
#include <stdio.h>
int main(int argc, char *argv[]) {
    if (argc != 3) {
        printf("Usage: %s num1 num2\n", argv[0]);
        return 1;
    }
    int a = atoi(argv[1]);
    int b = atoi(argv[2]);
    printf("Sum = %d\n", a + b);
    return 0;
}
```

```
$ ./program 15 25
```

```
Output
Sum = 40
```

Explanation: argv[1] and argv[2] are received as strings and converted to integers using atoi() before addition.

Chapter Summary — Programming Examples

- Practiced: basic pointer use, swap via call by reference, pointer arithmetic, array traversal, array of pointers, and command-line arguments.
- Each program demonstrates a real exam-style application of the corresponding theory topic.

TOPIC 16

Quick Revision

16.1 Important Definitions (One Line Each)

Term	One-line Definition
Pointer	A variable that stores the address of another variable
Address operator (&)	Returns the memory address of a variable
Indirection operator (*)	Accesses/modifies the value at the address a pointer holds
Wild pointer	An uninitialized pointer pointing to an unknown location
NULL pointer	A pointer explicitly set to point to nothing (address 0)
Void pointer	A generic pointer that can hold the address of any data type
Array of pointers	An array where each element is a pointer
argc / argv	Count and vector of command-line arguments passed to main()

16.2 Syntax Summary

QUICK SYNTAX SHEET

```
datatype *p;              // declare pointer
p = &var;                 // point to var
*p                        // access value at p
p + n / p - n             // pointer arithmetic
p++ / p--                 // move to next/prev element
arr == &arr[0]            // array name as pointer
*(arr + i) == arr[i]      // array-pointer equivalence
datatype *arr[size];      // array of pointers
int main(int argc, char *argv[])   // command-line arguments
```

16.3 Frequently Asked Exam Questions

1. What is a pointer? Why are pointers used in C?
2. Differentiate between & and * operators with examples.
3. Explain the relationship between arrays and pointers with an example.
4. What is pointer arithmetic? State its rules.
5. Write a program to swap two numbers using pointers (call by reference).
6. What is an array of pointers? How is it different from a 2-D array?
7. Explain argc and argv with a suitable example.
8. What is a wild pointer / dangling pointer? How can it be avoided?
9. Differentiate *p++ from (*p)++.
10. Why must sizeof(pointer) be same regardless of the type it points to?

16.4 One-Page Revision Notes

Core Rules

- Pointer holds an address, not a value.
- & gets address; * gets value at address.
- Array name = constant pointer to element 0.
- Pointer arithmetic scales by sizeof(type).

Must-Remember Facts

- argv[0] = program name; argv[argc] = NULL.
- sizeof(pointer) is fixed per system, not per type.
- Arrays passed to functions decay to pointers.
- Only integer +/- allowed on pointers, not pointer*pointer.

16.5 Common Mistakes and Debugging Tips

Mistake	Why It's Wrong	Fix

Using an uninitialized pointer	Points to a random/garbage address (wild pointer)	Always initialize: int *p = NULL;
Dereferencing a NULL pointer	Causes a crash (segmentation fault)	Check `if (p != NULL)` before dereferencing
Confusing *p++ with (*p)++	Different operator precedence effects	Use parentheses explicitly when in doubt
Assuming sizeof(arr)==sizeof(p) in functions	Arrays decay to pointers when passed as parameters	Always pass array length separately
Comparing unrelated pointers	Undefined behaviour unless pointers refer to same array	Only compare pointers within the same array/object
Forgetting argv values are strings	Cannot use numeric arguments directly in arithmetic	Convert using atoi()/atof()/strtol()

Final Exam Tip: When answering theory questions on pointers, always support your answer with (1) a one-line definition, (2) correct syntax, and (3) a small example with output — this scoring pattern is expected in most university and competitive exams.

End-of-Guide Summary

Pointers are variables that store memory addresses, enabling direct memory access, call by reference, dynamic memory management, and efficient array/string handling. Mastery of the & and * operators, pointer arithmetic, the array-pointer relationship, arrays of pointers, and command-line arguments (argc/argv) covers the complete syllabus scope required for exams.